===== FILE: drizzle/schema.ts =====

```
import {
bigint,
boolean,
index,
int,
mysqlEnum,
mysqlTable,
text,
timestamp,
uniqueIndex,
varchar,
} from "drizzle-orm/mysql-core";
import { ITEM_CATEGORIES, ITEM_STATUSES } from "../shared/lostFound";
export const users = mysqlTable(
"users",
{
id: int("id").autoincrement().primaryKey(),
openId: varchar("openId", { length: 64 }).notNull().unique(),
name: text("name"),
email: varchar("email", { length: 320 }),
loginMethod: varchar("loginMethod", { length: 64 }),
role: mysqlEnum("role", ["user", "admin"]).default("user").notNull(),
createdAt: timestamp("createdAt").defaultNow().notNull(),
updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
lastSignedIn: timestamp("lastSignedIn").defaultNow().notNull(),
},
table => [index("users_role_idx").on(table.role)],
);
export const localCredentials = mysqlTable(
"local_credentials",
{
```

```
id: int("id").autoincrement().primaryKey(),
userId: int("userId").notNull().references(() => users.id, { onDelete: "cascade" }),
email: varchar("email", { length: 320 }).notNull(),
studentId: varchar("studentId", { length: 32 }).notNull(),
passwordHash: varchar("passwordHash", { length: 512 }).notNull(),
createdAt: bigint("createdAt", { mode: "number" }).notNull(),
updatedAt: bigint("updatedAt", { mode: "number" }).notNull(),
},
table => [
uniqueIndex("local_credentials_user_unique").on(table.userId),
uniqueIndex("local_credentials_email_unique").on(table.email),
uniqueIndex("local_credentials_student_id_unique").on(table.studentId),
],
);
export const profiles = mysqlTable(
"profiles",
{
id: int("id").autoincrement().primaryKey(),
userId: int("userId").notNull().references(() => users.id, { onDelete: "cascade" }),
studentId: varchar("studentId", { length: 32 }).notNull(),
department: varchar("department", { length: 120 }).notNull(),
phone: varchar("phone", { length: 32 }).notNull(),
affiliation: mysqlEnum("affiliation", ["student", "staff"]).default("student").notNull(),
contactInfo: varchar("contactInfo", { length: 320 }).notNull(),
profileCompleted: boolean("profileCompleted").default(true).notNull(),
createdAt: bigint("createdAt", { mode: "number" }).notNull(),
updatedAt: bigint("updatedAt", { mode: "number" }).notNull(),
},
table => [
uniqueIndex("profiles_user_unique").on(table.userId),
uniqueIndex("profiles_student_id_unique").on(table.studentId),
index("profiles_affiliation_idx").on(table.affiliation),
```

```
],
);
export const items = mysqlTable(
"items",
{
id: int("id").autoincrement().primaryKey(),
reporterId: int("reporterId").notNull().references(() => users.id, { onDelete: "cascade" }),
reportType: mysqlEnum("reportType", ["lost", "found"]).notNull(),
title: varchar("title", { length: 160 }).notNull(),
description: text("description").notNull(),
searchText: varchar("searchText", { length: 512 }).notNull(),
category: mysqlEnum("category", ITEM_CATEGORIES).notNull(),
eventDate: bigint("eventDate", { mode: "number" }).notNull(),
location: varchar("location", { length: 220 }).notNull(),
holdingLocation: varchar("holdingLocation", { length: 220 }),
contactDetails: varchar("contactDetails", { length: 320 }).notNull(),
imageKey: varchar("imageKey", { length: 512 }),
imageUrl: varchar("imageUrl", { length: 768 }),
status: mysqlEnum("status", ITEM_STATUSES).default("Lost").notNull(),
adminNote: text("adminNote"),
returnedAt: bigint("returnedAt", { mode: "number" }),
createdAt: bigint("createdAt", { mode: "number" }).notNull(),
updatedAt: bigint("updatedAt", { mode: "number" }).notNull(),
},
table => [
index("items_reporter_idx").on(table.reporterId),
index("items_type_status_idx").on(table.reportType, table.status),
index("items_category_idx").on(table.category),
index("items_title_idx").on(table.title),
index("items_location_idx").on(table.location),
index("items_search_text_idx").on(table.searchText),
index("items_event_date_idx").on(table.eventDate),
```

```
index("items_created_at_idx").on(table.createdAt),
],
);
export const claims = mysqlTable(
"claims",
{
id: int("id").autoincrement().primaryKey(),
itemId: int("itemId").notNull().references(() => items.id, { onDelete: "cascade" }),
claimantId: int("claimantId").notNull().references(() => users.id, { onDelete: "cascade" }),
uniqueIdentifiers: text("uniqueIdentifiers").notNull(),
proofDescription: text("proofDescription").notNull(),
status: mysqlEnum("status", ["pending", "approved",
"rejected"]).default("pending").notNull(),
reviewerId: int("reviewerId").references(() => users.id, { onDelete: "set null" }),
reviewNote: text("reviewNote"),
createdAt: bigint("createdAt", { mode: "number" }).notNull(),
reviewedAt: bigint("reviewedAt", { mode: "number" }),
},
table => [
uniqueIndex("claims_item_claimant_unique").on(table.itemId, table.claimantId),
index("claims_item_status_idx").on(table.itemId, table.status),
index("claims_claimant_idx").on(table.claimantId),
index("claims_status_idx").on(table.status),
],
);
export const statusHistory = mysqlTable(
"status_history",
{
id: int("id").autoincrement().primaryKey(),
itemId: int("itemId").notNull().references(() => items.id, { onDelete: "cascade" }),
fromStatus: mysqlEnum("fromStatus", ITEM_STATUSES),
toStatus: mysqlEnum("toStatus", ITEM_STATUSES).notNull(),
```

```
actorId: int("actorId").notNull().references(() => users.id, { onDelete: "cascade" }),
note: text("note"),
createdAt: bigint("createdAt", { mode: "number" }).notNull(),
},
table => [index("status_history_item_idx").on(table.itemId, table.createdAt)],
);
export const itemMatches = mysqlTable(
"item_matches",
{
id: int("id").autoincrement().primaryKey(),
lostItemId: int("lostItemId").notNull().references(() => items.id, { onDelete: "cascade" }),
foundItemId: int("foundItemId").notNull().references(() => items.id, { onDelete: "cascade" }),
matchScore: int("matchScore").notNull(),
createdAt: bigint("createdAt", { mode: "number" }).notNull(),
},
table => [
uniqueIndex("item_matches_pair_unique").on(table.lostItemId, table.foundItemId),
index("item_matches_lost_idx").on(table.lostItemId),
index("item_matches_found_idx").on(table.foundItemId),
],
);
export const notifications = mysqlTable(
"notifications",
{
id: int("id").autoincrement().primaryKey(),
userId: int("userId").notNull().references(() => users.id, { onDelete: "cascade" }),
type: mysqlEnum("type", [
"item_match",
"claim_submitted",
"claim_approved",
"claim_rejected",
]).notNull(),
```

```
title: varchar("title", { length: 180 }).notNull(),
message: text("message").notNull(),
itemId: int("itemId").references(() => items.id, { onDelete: "set null" }),
claimId: int("claimId").references(() => claims.id, { onDelete: "set null" }),
isRead: boolean("isRead").default(false).notNull(),
createdAt: bigint("createdAt", { mode: "number" }).notNull(),
},
table => [
index("notifications_user_read_idx").on(table.userId, table.isRead, table.createdAt),
index("notifications_item_idx").on(table.itemId),
],
);
export const emailDeliveries = mysqlTable(
"email_deliveries",
{
id: int("id").autoincrement().primaryKey(),
notificationId: int("notificationId").notNull().references(() => notifications.id, { onDelete:
"cascade" }),
recipient: varchar("recipient", { length: 320 }).notNull(),
subject: varchar("subject", { length: 220 }).notNull(),
body: text("body").notNull(),
status: mysqlEnum("status", ["queued", "sent", "failed",
"skipped"]).default("queued").notNull(),
providerId: varchar("providerId", { length: 180 }),
failureReason: text("failureReason"),
createdAt: bigint("createdAt", { mode: "number" }).notNull(),
sentAt: bigint("sentAt", { mode: "number" }),
},
table => [
index("email_delivery_notification_idx").on(table.notificationId),
index("email_delivery_status_idx").on(table.status, table.createdAt),
],
```

);

export type User = typeof users.$inferSelect; $exporttypeInsertUser = typeofusers.$ inferInsert;

export type LocalCredential = typeof localCredentials. $inferSelect; $exporttypeProfile = typeofprofiles.$inferSelect;

export type Item = typeof items.$inferSelect; $exporttypeInsertItem = typeofitems.$ inferInsert;

export type Claim = typeof claims. $inferSelect; $exporttypeNotification = typeofnotifications.$inferSelect;

===== FILE: server/routers.ts =====

import { COOKIE_NAME } from "@shared/const";

import { getSessionCookieOptions } from "./_core/cookies";

import { systemRouter } from "./_core/systemRouter";

import { publicProcedure, router } from "./_core/trpc";

import { adminRouter } from "./routers/admin";

import { claimsRouter } from "./routers/claims";

import { itemsRouter } from "./routers/items";

import { localAuthRouter } from "./routers/localAuth";

import { mediaRouter } from "./routers/media";

import { notificationsRouter } from "./routers/notifications";

import { profileRouter } from "./routers/profile";

export const appRouter = router({

// if you need to use socket.io, read and register route in server/_core/index.ts, all api should start with '/api/' so that the gateway can route correctly

system: systemRouter,

auth: router({

me: publicProcedure.query(opts => opts.ctx.user),

logout: publicProcedure.mutation(({ ctx }) => {

const cookieOptions = getSessionCookieOptions(ctx.req);

ctx.res.clearCookie(COOKIE_NAME, { ...cookieOptions, maxAge: -1 });

return {

success: true,

} as const;

```
}),
}),
localAuth: localAuthRouter,
profile: profileRouter,
media: mediaRouter,
items: itemsRouter,
claims: claimsRouter,
notifications: notificationsRouter,
admin: adminRouter,
});
export type AppRouter = typeof appRouter;
===== FILE: server/routers/localAuth.ts =====
import { TRPCError } from "@trpc/server";
import { and, eq, or } from "drizzle-orm";
import type { Response } from "express";
import { nanoid } from "nanoid";
import { localCredentials, profiles, users } from "../../drizzle/schema";
import { manualLoginInput, manualRegistrationInput } from "../../shared/manualAuth";
import { COOKIE_NAME, ONE_YEAR_MS } from "../../shared/const";
import { getSessionCookieOptions } from "../_core/cookies";
import { sdk } from "../_core/sdk";
import { publicProcedure, router } from "../_core/trpc";
import { requireDb } from "../db";
import { hashPassword, verifyPassword } from "../services/passwords";
function setManualSession(res: Response | undefined, req: Parameters<typeof
getSessionCookieOptions>[0], token: string) {
if (!res) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR", message: "Unable to
create a secure session." });
res.cookie(COOKIE_NAME, token, { ...getSessionCookieOptions(req), maxAge:
ONE_YEAR_MS });
}
export const localAuthRouter = router({
```

register: publicProcedure.input(manualRegistrationInput).mutation(async ({ ctx, input }) => {

const db = await requireDb();

const [emailOwner, credentialOwner, profileOwner] = await Promise.all([

db.select({ id: users.id }).from(users).where(eq(users.email, input.email)).limit(1),

db.select({ id: localCredentials.id }).from(localCredentials).where(or(eq(localCredentials.email, input.email), eq(localCredentials.studentId, input.studentId))).limit(1),

db.select({ id: profiles.id }).from(profiles).where(eq(profiles.studentId, input.studentId)).limit(1),

]);

Plain Text

```
if (emailOwner[0] || credentialOwner[0] || profileOwner[0]) {
  throw new TRPCError({ code: "CONFLICT", message: "An account with that Gmail
}

const now = Date.now();
const passwordHash = await hashPassword(input.password);

try {
  const user = await db.transaction(async tx => {
    const inserted = await tx
      .insert(users)
      .values({
        openId: `local_${nanoid(28)}`,
        name: input.name,
        email: input.email,
        loginMethod: "password",
        role: "user",
        lastSignedIn: new Date(),
      })
      .$returningId();
    const userId = inserted[0]?.id;
    if (!userId) throw new Error("Unable to create account.");

    await tx.insert(localCredentials).values({
      userId,
      email: input.email,
      studentId: input.studentId,
      passwordHash,
      createdAt: now,
      updatedAt: now,
```

```
    });

    return { id: userId };
  });

  const registeredUser = await db.select().from(users).where(eq(users.id, user..
  const account = registeredUser[0];
  if (!account) throw new Error("Unable to load the new account.");
  const token = await sdk.createSessionToken(account.openId, { name: account.na
  setManualSession(ctx.res, ctx.req, token);
  return { success: true, requiresProfile: true, user: { id: account.id, name: 
} catch (error) {
  if (error instanceof TRPCError) throw error;
  if (error instanceof Error && error.message.toLowerCase().includes("duplicate
    throw new TRPCError({ code: "CONFLICT", message: "An account with that Gmai
  }
  throw error;
}
```

}),

login: publicProcedure.input(manualLoginInput).mutation(async ({ ctx, input }) => {

const db = await requireDb();

const result = await db

.select({ credential: localCredentials, user: users })

.from(localCredentials)

.innerJoin(users, eq(users.id, localCredentials.userId))

.where(and(eq(localCredentials.email, input.email), eq(users.loginMethod, "password")))

.limit(1);

const account = result[0];

const valid = account ? await verifyPassword(input.password, account.credential.passwordHash) : false;

if (!account || !valid) {

throw new TRPCError({ code: "UNAUTHORIZED", message: "Invalid Gmail address or password." });

}

Plain Text

```
await db.update(users).set({ lastSignedIn: new Date() }).where(eq(users.id, acc
const token = await sdk.createSessionToken(account.user.openId, { name: account
setManualSession(ctx.res, ctx.req, token);
```

```
  const profile = await db.select({ id: profiles.id }).from(profiles).where(eq(pr
  return {
    success: true,
    requiresProfile: !profile[0],
    user: { id: account.user.id, name: account.user.name, email: account.user.ema
  };
```

}),

});

===== FILE: server/routers/items.ts =====

import { TRPCError } from "@trpc/server";

import {

and,

asc,

count,

desc,

eq,

gte,

like,

lte,

ne,

or,

type SQL,

} from "drizzle-orm";

import { z } from "zod";

import { claims, itemMatches, items, profiles, statusHistory, users } from "../../drizzle/schema";

import { ITEM_CATEGORIES, ITEM_STATUSES } from "../../shared/lostFound";

import { protectedProcedure, publicProcedure, router } from "../_core/trpc";

import { requireDb } from "../db";

import { calculateMatchScore, isLikelyMatch } from "../services/matching";

import { dispatchNotification } from "../services/notifications";

import { canDeleteItem, canEditItem } from "../services/rules";

const reportInput = z

```
.object({
reportType: z.enum(["lost", "found"]),
title: z.string().trim().min(3).max(160),
description: z.string().trim().min(12).max(4_000),
category: z.enum(ITEM_CATEGORIES),
eventDate: z.number().int().positive(),
location: z.string().trim().min(3).max(220),
holdingLocation: z.string().trim().max(220).optional().nullable(),
contactDetails: z.string().trim().min(4).max(320),
imageKey: z.string().max(512).optional().nullable(),
imageUrl: z.string().max(768).optional().nullable(),
})
.superRefine((value, ctx) => {
if (value.eventDate > Date.now() + 86_400_000) {
ctx.addIssue({ code: "custom", path: ["eventDate"], message: "The event date cannot be in the future." });
}
if (value.reportType === "found" && !value.holdingLocation?.trim()) {
ctx.addIssue({ code: "custom", path: ["holdingLocation"], message: "A holding location is required." });
}
});
function buildSearchText(title: string, description: string, location: string) {
return `${title} ${description} ${location}`
.toLowerCase()
.replace(/[^a-z0-9\s-]/g, " ")
.replace(/\s+/g, " ")
.trim()
.slice(0, 512);
}
async function requireCompletedProfile(userId: number) {
const db = await requireDb();
```

```
const result = await db.select({ id: profiles.id }).from(profiles).where(eq(profiles.userId, userId)).limit(1);
if (!result[0]) {
throw new TRPCError({ code: "PRECONDITION_FAILED", message: "Complete your DIU profile before submitting a report." });
}
}
async function detectMatches(itemId: number) {
const db = await requireDb();
const sourceRows = await db
.select({ item: items, reporterEmail: users.email })
.from(items)
.innerJoin(users, eq(items.reporterId, users.id))
.where(eq(items.id, itemId))
.limit(1);
const source = sourceRows[0];
if (!source) return;
const complementaryType = source.item.reportType === "found" ? "lost" : "found";
const candidates = await db
.select({ item: items, reporterEmail: users.email })
.from(items)
.innerJoin(users, eq(items.reporterId, users.id))
.where(
and(
eq(items.reportType, complementaryType),
eq(items.category, source.item.category),
ne(items.status, "Returned"),
ne(items.reporterId, source.item.reporterId),
),
)
.limit(50);
for (const candidate of candidates) {
```

const lost = source.item.reportType === "lost" ? source.item : candidate.item;
const found = source.item.reportType === "found" ? source.item : candidate.item;
if (!isLikelyMatch(lost, found)) continue;
const existing = await db
.select({ id: itemMatches.id })
.from(itemMatches)
.where(and(eq(itemMatches.lostItemId, lost.id), eq(itemMatches.foundItemId, found.id)))
.limit(1);
if (existing[0]) continue;

Plain Text

```
const score = calculateMatchScore(lost, found);
await db.insert(itemMatches).values({
  lostItemId: lost.id,
  foundItemId: found.id,
  matchScore: score,
  createdAt: Date.now(),
});
const lostReporterEmail = source.item.reportType === "lost" ? source.reporterEm
await dispatchNotification({
  userId: lost.reporterId,
  recipientEmail: lostReporterEmail,
  type: "item_match",
  title: "A possible match was found",
  message: `A found item may match “${lost.title}”. Review the details before s
  itemId: found.id,
});
```

}
}
export const itemsRouter = router({
list: publicProcedure
.input(
z
.object({
search: z.string().trim().max(100).default(""),
reportType: z.enum(["all", "lost", "found"]).default("all"),
category: z.enum(["all", ...ITEM_CATEGORIES]).default("all"),

```
status: z.enum(["all", ...ITEM_STATUSES]).default("all"),
dateFrom: z.number().int().positive().optional(),
dateTo: z.number().int().positive().optional(),
page: z.number().int().min(1).default(1),
pageSize: z.number().int().min(1).max(24).default(9),
sort: z.enum(["newest", "oldest", "eventDate"]).default("newest"),
})
.optional(),
)
.query(async ({ input }) => {
const db = await requireDb();
const filters = input ?? {
search: "",
reportType: "all" as const,
category: "all" as const,
status: "all" as const,
page: 1,
pageSize: 9,
sort: "newest" as const,
};
const conditions: SQL[] = [];
if (filters.search) {
const normalized = filters.search.toLowerCase().replace(/[^a-z0-9\s-]/g, " ").replace(/\s+/g,
" ").trim();
const query = %${normalized}% ;
conditions.push(
or(like(items.searchText, query), like(items.title, query), like(items.location, query))!,
);
}
if (filters.reportType !== "all") conditions.push(eq(items.reportType, filters.reportType));
if (filters.category !== "all") conditions.push(eq(items.category, filters.category));
if (filters.status !== "all") conditions.push(eq(items.status, filters.status));
```

if (filters.dateFrom) conditions.push(gte(items.eventDate, filters.dateFrom));

if (filters.dateTo) conditions.push(lte(items.eventDate, filters.dateTo));

const where = conditions.length ? and(...conditions) : undefined;

const orderBy =

filters.sort === "oldest"

? asc(items.createdAt)

: filters.sort === "eventDate"

? desc(items.eventDate)

: desc(items.createdAt);

Plain Text

```
  const [rows, totalRows] = await Promise.all([
    db
      .select({
        id: items.id,
        reporterId: items.reporterId,
        reporterName: users.name,
        reportType: items.reportType,
        title: items.title,
        description: items.description,
        category: items.category,
        eventDate: items.eventDate,
        location: items.location,
        holdingLocation: items.holdingLocation,
        contactDetails: items.contactDetails,
        imageUrl: items.imageUrl,
        status: items.status,
        createdAt: items.createdAt,
        updatedAt: items.updatedAt,
      })
      .from(items)
      .innerJoin(users, eq(items.reporterId, users.id))
      .where(where)
      .orderBy(orderBy)
      .limit(filters.pageSize)
      .offset((filters.page - 1) * filters.pageSize),
    db.select({ value: count() }).from(items).where(where),
  ]);
  const total = totalRows[0]?.value ?? 0;
  return {
    items: rows,
    total,
    page: filters.page,
```

```
    pageSize: filters.pageSize,
    totalPages: Math.max(1, Math.ceil(total / filters.pageSize)),
  };
}),
```

detail: publicProcedure.input(z.object({ id: z.number().int().positive() })).query(async ({ ctx, input }) => {

const db = await requireDb();

const result = await db

.select({ item: items, reporterName: users.name })

.from(items)

.innerJoin(users, eq(items.reporterId, users.id))

.where(eq(items.id, input.id))

.limit(1);

if (!result[0]) throw new TRPCError({ code: "NOT_FOUND", message: "Item report not found." });

Plain Text

```
const history = await db
  .select()
  .from(statusHistory)
  .where(eq(statusHistory.itemId, input.id))
  .orderBy(asc(statusHistory.createdAt));
const ownClaim = ctx.user
  ? (
      await db
        .select()
        .from(claims)
        .where(and(eq(claims.itemId, input.id), eq(claims.claimantId, ctx.user.
        .limit(1)
    )[0] ?? null
  : null;
return {
  ...result[0],
  history,
  ownClaim,
  canManage: Boolean(ctx.user && (ctx.user.id === result[0].item.reporterId ||
};
```

}),

mine: protectedProcedure.query(async ({ ctx }) => {

```
const db = await requireDb();
return db.select().from(items).where(eq(items.reporterId,
ctx.user.id)).orderBy(desc(items.createdAt));
}),
create: protectedProcedure.input(reportInput).mutation(async ({ ctx, input }) => {
await requireCompletedProfile(ctx.user.id);
const db = await requireDb();
const now = Date.now();
const inserted = await db
.insert(items)
.values({
reporterId: ctx.user.id,
reportType: input.reportType,
title: input.title,
description: input.description,
searchText: buildSearchText(input.title, input.description, input.location),
category: input.category,
eventDate: input.eventDate,
location: input.location,
holdingLocation: input.reportType === "found" ? input.holdingLocation : null,
contactDetails: input.contactDetails,
imageKey: input.imageKey ?? null,
imageUrl: input.imageUrl ?? null,
status: "Lost",
createdAt: now,
updatedAt: now,
})
.$returningId();
const itemId = inserted[0]?.id;
if (!itemId) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR", message: "Unable to
create the report." });
await db.insert(statusHistory).values({
itemId,
```

```
fromStatus: null,
toStatus: "Lost",
actorId: ctx.user.id,
note: input.reportType === "lost" ? "Lost item report created." : "Found item report created.",
createdAt: now,
});
await detectMatches(itemId);
return { id: itemId };
}),
update: protectedProcedure
.input(reportInput.omit({ reportType: true }).partial().extend({ id: z.number().int().positive() }))
.mutation(async ({ ctx, input }) => {
const db = await requireDb();
const existing = await db.select().from(items).where(eq(items.id, input.id)).limit(1);
const item = existing[0];
if (!item) throw new TRPCError({ code: "NOT_FOUND", message: "Item report not found." });
if (!canEditItem(item, ctx.user)) {
throw new TRPCError({
code: item.reporterId === ctx.user.id ? "CONFLICT" : "FORBIDDEN",
message: item.reporterId === ctx.user.id ? "Only open reports can be edited." : "You cannot edit this report.",
});
}
const { id, ...changes } = input;
const nextTitle = changes.title ?? item.title;
const nextDescription = changes.description ?? item.description;
const nextLocation = changes.location ?? item.location;
await db
.update(items)
.set({
...changes,
```

```
searchText: buildSearchText(nextTitle, nextDescription, nextLocation),
updatedAt: Date.now(),
})
.where(eq(items.id, id));
return { success: true } as const;
}),
delete: protectedProcedure.input(z.object({ id: z.number().int().positive()
})).mutation(async ({ ctx, input }) => {
const db = await requireDb();
const existing = await db.select().from(items).where(eq(items.id, input.id)).limit(1);
const item = existing[0];
if (!item) throw new TRPCError({ code: "NOT_FOUND", message: "Item report not found." });
if (!canDeleteItem(item, ctx.user)) {
throw new TRPCError({
code: item.reporterId === ctx.user.id ? "CONFLICT" : "FORBIDDEN",
message:
item.reporterId === ctx.user.id
? "A report with an active claim cannot be deleted."
: "You cannot delete this report.",
});
}
await db.delete(items).where(eq(items.id, input.id));
return { success: true } as const;
}),
});
===== FILE: server/routers/claims.ts =====
import { TRPCError } from "@trpc/server";
import { and, desc, eq } from "drizzle-orm";
import { z } from "zod";
import { claims, items, profiles, statusHistory, users } from "../../drizzle/schema";
import { protectedProcedure, router } from "../_core/trpc";
import { requireDb } from "../db";
```

```
import { dispatchNotification } from "../services/notifications";
import { claimSubmissionIssue, statusAfterClaimSubmission } from "../services/rules";
export const claimsRouter = router({
mine: protectedProcedure.query(async ({ ctx }) => {
const db = await requireDb();
return db
.select({
claim: claims,
itemTitle: items.title,
itemStatus: items.status,
itemImageUrl: items.imageUrl,
})
.from(claims)
.innerJoin(items, eq(claims.itemId, items.id))
.where(eq(claims.claimantId, ctx.user.id))
.orderBy(desc(claims.createdAt));
}),
submit: protectedProcedure
.input(
z.object({
itemId: z.number().int().positive(),
uniqueIdentifiers: z.string().trim().min(6).max(2_000),
proofDescription: z.string().trim().min(12).max(4_000),
}),
)
.mutation(async ({ ctx, input }) => {
const db = await requireDb();
const profile = await db.select({ id: profiles.id }).from(profiles).where(eq(profiles.userId, ctx.user.id)).limit(1);
if (!profile[0]) {
throw new TRPCError({ code: "PRECONDITION_FAILED", message: "Complete your DIU profile before submitting a claim." });
```

}

Plain Text

```
  const itemRows = await db
    .select({ item: items, reporterEmail: users.email })
    .from(items)
    .innerJoin(users, eq(items.reporterId, users.id))
    .where(eq(items.id, input.itemId))
    .limit(1);
  const row = itemRows[0];
  if (!row) throw new TRPCError({ code: "NOT_FOUND", message: "Found item repor
  const duplicate = await db
    .select({ id: claims.id })
    .from(claims)
    .where(and(eq(claims.itemId, input.itemId), eq(claims.claimantId, ctx.user.
    .limit(1);
  const issue = claimSubmissionIssue(row.item, ctx.user.id, Boolean(duplicate[0
  if (issue) {
    throw new TRPCError({
      code: row.item.status === "Verified" || row.item.status === "Returned" ||
      message: issue,
    });
  }

  const now = Date.now();
  const inserted = await db
    .insert(claims)
    .values({
      itemId: input.itemId,
      claimantId: ctx.user.id,
      uniqueIdentifiers: input.uniqueIdentifiers,
      proofDescription: input.proofDescription,
      status: "pending",
      createdAt: now,
    })
    .$returningId();
  const claimId = inserted[0]?.id;
  if (!claimId) throw new TRPCError({ code: "INTERNAL_SERVER_ERROR", message: "

  const nextStatus = statusAfterClaimSubmission(row.item.status);
  if (nextStatus !== row.item.status) {
    await db.update(items).set({ status: nextStatus, updatedAt: now }).where(eq
    await db.insert(statusHistory).values({
      itemId: row.item.id,
      fromStatus: "Lost",
      toStatus: "Claimed",
      actorId: ctx.user.id,
```

```
      note: "An ownership claim was submitted.",
      createdAt: now,
    });
  }

  await dispatchNotification({
    userId: row.item.reporterId,
    recipientEmail: row.reporterEmail,
    type: "claim_submitted",
    title: "A new ownership claim was submitted",
    message: `A DIU user submitted an ownership claim for “${row.item.title}”. S
    itemId: row.item.id,
    claimId,
  });
  return { id: claimId };
}),
```

});

===== FILE: server/routers/admin.ts =====

import { TRPCError } from "@trpc/server";

import { and, asc, count, desc, eq, ne } from "drizzle-orm";

import { z } from "zod";

import { claims, items, statusHistory, users } from "../../drizzle/schema";

import { adminProcedure, router } from "../_core/trpc";

import { requireDb } from "../db";

import { dispatchNotification } from "../services/notifications";

import { statusAfterClaimReview } from "../services/rules";

export const adminRouter = router({

overview: adminProcedure.query(async () => {

const db = await requireDb();

const [itemCounts, claimCounts, userCount] = await Promise.all([

db.select({ status: items.status, value: count() }).from(items).groupBy(items.status),

db.select({ status: claims.status, value: count() }).from(claims).groupBy(claims.status),

db.select({ value: count() }).from(users),

]);

const byItemStatus = Object.fromEntries(itemCounts.map(entry => [entry.status, entry.value]));

```
const byClaimStatus = Object.fromEntries(claimCounts.map(entry => [entry.status, entry.value]));
return {
totalUsers: userCount[0]?.value ?? 0,
lost: byItemStatus.Lost ?? 0,
claimed: byItemStatus.Claimed ?? 0,
verified: byItemStatus.Verified ?? 0,
returned: byItemStatus.Returned ?? 0,
pendingClaims: byClaimStatus.pending ?? 0,
};
}),
claims: adminProcedure
.input(
z
.object({
status: z.enum(["all", "pending", "approved", "rejected"]).default("pending"),
page: z.number().int().min(1).default(1),
pageSize: z.number().int().min(1).max(50).default(20),
})
.optional(),
)
.query(async ({ input }) => {
const db = await requireDb();
const filters = input ?? { status: "pending" as const, page: 1, pageSize: 20 };
const where = filters.status === "all" ? undefined : eq(claims.status, filters.status);
const rows = await db
.select({
claim: claims,
item: items,
claimantName: users.name,
claimantEmail: users.email,
})
```

```
.from(claims)
.innerJoin(items, eq(claims.itemId, items.id))
.innerJoin(users, eq(claims.claimantId, users.id))
.where(where)
.orderBy(desc(claims.createdAt))
.limit(filters.pageSize)
.offset((filters.page - 1) * filters.pageSize);
const totalRows = await db.select({ value: count() }).from(claims).where(where);
const total = totalRows[0]?.value ?? 0;
return { rows, total, totalPages: Math.max(1, Math.ceil(total / filters.pageSize)) };
}),
reviewClaim: adminProcedure
.input(
z.object({
claimId: z.number().int().positive(),
decision: z.enum(["approved", "rejected"]),
reviewNote: z.string().trim().min(4).max(2_000),
}),
)
.mutation(async ({ ctx, input }) => {
const db = await requireDb();
const rows = await db
.select({ claim: claims, item: items, claimantEmail: users.email })
.from(claims)
.innerJoin(items, eq(claims.itemId, items.id))
.innerJoin(users, eq(claims.claimantId, users.id))
.where(eq(claims.id, input.claimId))
.limit(1);
const row = rows[0];
if (!row) throw new TRPCError({ code: "NOT_FOUND", message: "Claim not found." });
if (row.claim.status !== "pending") {
```

throw new TRPCError({ code: "CONFLICT", message: "This claim has already been reviewed." });

}

if (row.item.status === "Returned") {

throw new TRPCError({ code: "CONFLICT", message: "The item has already been returned." });

}

Plain Text

```
  const now = Date.now();
  await db
    .update(claims)
    .set({
      status: input.decision,
      reviewerId: ctx.user.id,
      reviewNote: input.reviewNote,
      reviewedAt: now,
    })
    .where(eq(claims.id, input.claimId));

  if (input.decision === "approved") {
    const competing = await db
      .select({ id: claims.id, claimantId: claims.claimantId, email: users.emai
      .from(claims)
      .innerJoin(users, eq(claims.claimantId, users.id))
      .where(and(eq(claims.itemId, row.item.id), eq(claims.status, "pending"),
    if (competing.length) {
      await db
        .update(claims)
        .set({ status: "rejected", reviewerId: ctx.user.id, reviewNote: "Anothe
        .where(and(eq(claims.itemId, row.item.id), eq(claims.status, "pending")
      for (const other of competing) {
        await dispatchNotification({
          userId: other.claimantId,
          recipientEmail: other.email,
          type: "claim_rejected",
          title: "Ownership claim rejected",
          message: `Your ownership claim for “${row.item.title}” was not approv
          itemId: row.item.id,
          claimId: other.id,
        });
      }
    }
    const fromStatus = row.item.status;
    const nextStatus = statusAfterClaimReview(fromStatus, "approved", 0);
```

```
    await db.update(items).set({ status: nextStatus, updatedAt: now }).where(eq
    await db.insert(statusHistory).values({
      itemId: row.item.id,
      fromStatus,
      toStatus: nextStatus,
      actorId: ctx.user.id,
      note: input.reviewNote,
      createdAt: now,
    });
  } else {
    const pending = await db
      .select({ value: count() })
      .from(claims)
      .where(and(eq(claims.itemId, row.item.id), eq(claims.status, "pending")))
    const nextStatus = statusAfterClaimReview(row.item.status, "rejected", pend
    if (nextStatus !== row.item.status) {
      await db.update(items).set({ status: nextStatus, updatedAt: now }).where(
      await db.insert(statusHistory).values({
        itemId: row.item.id,
        fromStatus: "Claimed",
        toStatus: nextStatus,
        actorId: ctx.user.id,
        note: "Claim rejected; the item is open for new claims.",
        createdAt: now,
      });
    }
  }

  await dispatchNotification({
    userId: row.claim.claimantId,
    recipientEmail: row.claimantEmail,
    type: input.decision === "approved" ? "claim_approved" : "claim_rejected",
    title: input.decision === "approved" ? "Ownership claim approved" : "Owners
    message:
      input.decision === "approved"
        ? `DIU staff verified your ownership claim for “${row.item.title}”. Ple
        : `DIU staff reviewed and rejected your ownership claim for “${row.item
    itemId: row.item.id,
    claimId: row.claim.id,
  });
  return { success: true } as const;
}),
```

markReturned: adminProcedure

.input(z.object({ itemId: z.number().int().positive(), note: z.string().trim().min(4).max(2_000) }))

.mutation(async ({ ctx, input }) => {

```
const db = await requireDb();
const rows = await db.select().from(items).where(eq(items.id, input.itemId)).limit(1);
const item = rows[0];
if (!item) throw new TRPCError({ code: "NOT_FOUND", message: "Item report not found." });
if (item.status !== "Verified") {
throw new TRPCError({ code: "CONFLICT", message: "Only a verified item can be marked as returned." });
}
const now = Date.now();
await db
.update(items)
.set({ status: "Returned", returnedAt: now, updatedAt: now, adminNote: input.note })
.where(eq(items.id, input.itemId));
await db.insert(statusHistory).values({
itemId: input.itemId,
fromStatus: "Verified",
toStatus: "Returned",
actorId: ctx.user.id,
note: input.note,
createdAt: now,
});
return { success: true } as const;
}),
recentReturns: adminProcedure.query(async () => {
const db = await requireDb();
return db
.select()
.from(items)
.where(eq(items.status, "Returned"))
.orderBy(desc(items.returnedAt))
.limit(10);
}),
```

```
});
```